

System Documentation

ResumePilot

AI-Powered Resume Reviewer & Job Application Tracker

Architecture Summary

ResumePilot is a modern full-stack web application designed to help job seekers optimize their resumes using AI and track their job application pipeline. It is built on the Next.js App Router framework, employing PostgreSQL for persistence, Prisma for database ORM, Clerk for secure identity management, Supabase Storage for resume files, Inngest for background step-function queues, and the Google Gemini API for structural parsing validation and constructive review critique.

Author:	Engineering & Architecture Team
Stack:	Next.js 16 (App Router), React 19, TypeScript, Prisma, Clerk, Supabase, Inngest, Gemini API
Document Version:	1.0.0
Date:	June 2026

1. System Architecture

ResumePilot utilizes a decoupled, event-driven serverless design. Users interact with a responsive single-page dashboard built with React 19. Core API routes manage user state, resume metadata, and job trackers synchronously, while heavy workloads—such as resume parsing (PDF/DOCX extraction) and generative AI feedback evaluations—are dispatched asynchronously to Inngest background queues.

High-Level System Topology

- **Client Layer:** React 19 Client Components, styled with Tailwind CSS, using TanStack React Query for declarative server-state management, cache invalidation, and optimistic state transitions.
- **Identity Provider:** Clerk Authentication middleware guarding frontend pages, Next.js Middleware routes, and API endpoints, mapping authentication states to the local database schema via standard Clerk Sync webhooks.
- **Storage Layer:** Supabase Object Storage API, accessed via secure Signed URL protocols, enabling controlled document storage for resumes.
- **Persistent Database:** PostgreSQL database mapped via Prisma Client, supporting high-concurrency connection pools and direct schema migrations.
- **Asynchronous Job Queue:** Inngest event processor orchestrating multi-step, durable workflows for resume analysis with state check-pointing.
- **AI Intelligence Layer:** Google Gemini Large Language Model (specifically utilizing `gemini-1.5-flash` or `gemini-3-flash-preview`) producing structured, schema-validated critiques.

Core Benefit of Inngest Event Pipeline

By decoupling the file parsing and LLM API calls from the client request-response loop, ResumePilot completely avoids API timeouts (common in serverless functions limited to 15s-30s execution boundaries) and provides robust automatic retry mechanisms on transient third-party API failures.

2. Database Schema Design

The application database uses a PostgreSQL instance managed via Prisma ORM. The relational model stores user profiles, uploaded documents, parse summaries, AI generated feedback reports, and tracked job application cycles.

Entity Relationship Models

```
// schema.prisma

enum ResumeStatus {
  UPLOADED
  PARSING
  REVIEWING
  READY
  FAILED
}

enum JobStatus {
  Saved
  Applied
  Interview
  Offer
  Rejected
}

model User {
  id          String    @id @default(uuid())
  clerkId     String    @unique
  name        String?
  email       String    @unique
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
  resumes     Resume[]
  jobs        Job[]
}

model Resume {
  id          String    @id @default(uuid())
  userId      String
  fileName    String
  storageKey  String
  mimeType    String
  size        Int
  roleTarget  String?
  targetLevel String?
  status      ResumeStatus @default(UPLOADED)
  createdAt   DateTime    @default(now())
  updatedAt   DateTime    @updatedAt

  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  parse     ResumeParse?
}
```

```

review      ResumeReview?
reviewHistory ResumeReviewHistory[]

@@index([userId, createdAt])
}

model ResumeParse {
  id          String    @id @default(uuid())
  resumeId    String    @unique
  rawText     String
  structuredJson Json
  parserVersion String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  resume      Resume    @relation(fields: [resumeId], references: [id], onDelete: Cascade)
}

model ResumeReview {
  id          String    @id @default(uuid())
  resumeId    String    @unique
  score       Int
  summaryJson Json
  missingKeywords Json
  suggestionsJson Json
  model       String
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  resume      Resume    @relation(fields: [resumeId], references: [id], onDelete: Cascade)
}

model ResumeReviewHistory {
  id          String    @id @default(uuid())
  resumeId    String
  score       Int
  summaryJson Json
  missingKeywords Json
  suggestionsJson Json
  model       String
  createdAt   DateTime @default(now())

  resume      Resume    @relation(fields: [resumeId], references: [id], onDelete: Cascade)

  @@index([resumeId, createdAt])
}

model Job {
  id          String    @id @default(uuid())
  userId      String
  company     String
  role        String
  status      JobStatus @default(Saved)
  notes       String?

```

```
followUp      String?
contactName   String?
contactEmail  String?
interviewRounds Json?
location      String?
link          String?
createdAt     DateTime @default(now())
updatedAt     DateTime @updatedAt

user          User      @relation(fields: [userId], references: [id], onDelete: Cascade)

@@index([userId, createdAt])
@@index([userId, status])
}
```

3. The Resume Parsing & AI Critique Pipeline

When a user uploads a resume, the document proceeds through a highly structured, automated processing chain. This cycle is fully asynchronous and coordinated via durable execution.

Processing Lifecycle

Phase	Status Code	Responsible Module	Action Performed
1. Upload	UPLOADED	<code>app/api/resumes/upload</code>	Validates file mime-type and file size limit. Uploads binary data to Supabase Storage, creates a database <code>Resume</code> record, and dispatches the <code>resume/uploaded</code> event to Inngest.
2. Parser	PARSING	<code>lib/resume-parser.ts</code>	Downloads the document from Supabase. Selects appropriate parser: <code>pdf-parse</code> for PDF binaries, or <code>mammoth</code> for Microsoft Word (DOCX) files. Normalizes spacing, separates lines, and extracts text structures.
3. Reviewer	REVIEWING	<code>lib/ai/review-resume.ts</code>	Generates a target prompt based on targeted role/seniority. Transmits the raw text and structure map to the Google Gemini API with instruction to output strict, structured JSON. Sanitizes response.
4. Ready	READY	<code>lib/inngest/functions</code>	Saves final critique properties into <code>ResumeReview</code> , appends a version history node to <code>ResumeReviewHistory</code> , and marks the resume as ready for front-end rendering.
5. Failed	FAILED	Global Error Catch Block	Catches network connection timeouts, parsing errors, or API schema violations, changing the target resume status to <code>FAILED</code> and logging execution logs.

Text Normalization and Section Segmentation

To feed clean, context-focused content into the Gemini LLM model and maximize assessment quality, the parser parses file buffers into plaintext strings and runs the following normalizations:

- Normalizes carriage returns (`\r\n`) to Unix newlines (`\n`), strips null bytes (`\u0000`), and removes tab whitespace.

- Uses aliases lookup mapping to identify section boundaries (e.g. mapping "Profile", "About", and "Professional Summary" into the single logical key `summary`).
- Applies regular expressions to distinguish between project lines, technical skills strings, educational structures, and personal contact lines.

AI Integration Details

The Gemini model evaluation prompt sets strict constraints for resume evaluation:

```
// lib/ai/review-resume.ts Prompt Constraints
- Provide 3-6 strengths
- Provide 3-6 weaknesses
- Provide 5-12 missingKeywords
- Provide exactly 3 rewriteSuggestions (each with before/after/why properties)
- Provide exactly 3 nextActions
- Score must reflect ATS readability + role alignment + measurable impact
```

The code requests Gemini output using `responseMimeType: "application/json"` to guarantee structural parseability. The returning string is filtered, validated, and normalized back into type-safe JavaScript objects.

4. REST API Endpoint Reference

All HTTP API endpoints are mounted within Next.js Route Handlers. Requests are authenticated using Clerk, ensuring users can only read and write data belonging to their respective identity record.

Resume Endpoints

GET /api/resumes

Lists resumes uploaded by the authenticated user. Supports filtering and search parameters.

- **Query Parameters:**

- **q** (string): Matches search queries against the filename, target role, or level.
- **status** (ResumeStatus): Filter files matching specific statuses.
- **dateRange** (string): Restricts search bounds to **today**, **7d**, or **30d**.

- **Success Response (200 OK):**

```
{
  "resumes": [
    {
      "id": "e581db32-dfc7-4eb7-a068-07e3bd26ef04",
      "fileName": "Aryan_Dahiya_Resume.pdf",
      "roleTarget": "Frontend Architect",
      "targetLevel": "Senior",
      "status": "READY",
      "createdAt": "2026-06-02T15:30:00.000Z",
      "score": 85
    }
  ],
  "totalCount": 1
}
```

POST /api/resumes/upload

Accepts a multipart file upload payload to start the resume parsing pipeline.

- **Request Body:** **FormData** containing:

- **file** (File): Binary PDF/DOCX file stream (Max 5MB).
- **roleTarget** (string): Targeted role description (e.g., "Fullstack Developer").
- **targetLevel** (string, optional): Seniority level (e.g., "Mid-Level").

- **Success Response (201 Created):**

```
{
  "resume": {
    "id": "e581db32-dfc7-4eb7-a068-07e3bd26ef04",
    "fileName": "Aryan_Dahiya_Resume.pdf",
    "roleTarget": "Frontend Architect",
  }
}
```

```

    "targetLevel": "Senior",
    "status": "UPLOADED",
    "size": 1204523,
    "mimeType": "application/pdf",
    "storageKey": "user_id/1770900057_Aryan_Dahiya_Resume.pdf",
    "bucket": "resumes",
    "createdAt": "2026-06-02T20:54:34.000Z"
  },
  "processingQueued": true
}

```

GET /api/resumes/[id]

Fetches full parsing information, current feedback review ratings, and historic review versions for a specific resume resource.

- **Success Response (200 OK):**

```

{
  "resume": {
    "id": "e581db32-dfc7-4eb7-a068-07e3bd26ef04",
    "fileName": "Aryan_Dahiya_Resume.pdf",
    "roleTarget": "Frontend Architect",
    "targetLevel": "Senior",
    "status": "READY",
    "createdAt": "2026-06-02T15:30:00.000Z",
    "updatedAt": "2026-06-02T15:31:12.000Z",
    "mimeType": "application/pdf",
    "size": 1204523,
    "storageKey": "user_id/1770900057_Aryan_Dahiya_Resume.pdf",
    "parse": {
      "parserVersion": "real-v1-pdfparse-mammoth",
      "rawText": "...",
      "structuredJson": {
        "summary": "...",
        "experience": ["..."],
        "projects": ["..."],
        "skills": ["..."],
        "education": ["..."]
      }
    },
    "feedback": {
      "score": 85,
      "strengths": ["Clear section headings", "Strong impact action verbs"],
      "weaknesses": ["Missing Kubernetes keyword", "Project metrics could be quantifiable"],
      "nextActions": ["Add quantifiable metrics to project items", "Include CI/CD skills"],
      "missingKeywords": ["Docker", "Kubernetes", "GraphQL"],
      "atsChecks": [],
      "rewriteSuggestions": [
        {
          "before": "Worked on the frontend code",
          "after": "Refactored user dashboard interface using Next.js, boosting load speed by 35%",
          "why": "Demonstrates impact, specific tech stack, and quantifies performance"
        }
      ]
    }
  }
}

```

```
gains."  
    }  
  ]  
},  
  "reviewHistory": []  
}  
}
```

Job Endpoints

GET /api/jobs

Retrieves a paginated list of job applications tracked by the active user account.

- **Query Parameters:**

- **q**: Filters jobs by company name, role title, or location fields.
- **status**: Restricts results to specific job status types.
- **page** (default 1): The page index.
- **limit** (default 10): Page size (maximum of 50).

- **Success Response (200 OK):**

```
{
  "jobs": [
    {
      "id": "a90412e8-d7a8-4221-a3f1-b92ccfe98c21",
      "company": "Google",
      "role": "Frontend Engineer",
      "status": "Interview",
      "location": "Bengaluru, India",
      "link": "https://careers.google.com/jobs",
      "createdAt": "2026-06-01T10:00:00.000Z",
      "interviewRounds": [
        { "name": "Technical Assessment", "status": "Done" },
        { "name": "System Design", "status": "Upcoming" }
      ]
    }
  ],
  "totalCount": 10,
  "filteredCount": 1,
  "page": 1,
  "limit": 10,
  "totalPages": 1
}
```

POST /api/jobs

Creates a new tracked job application record in the database. Schema-validated via **zod**.

- **Request Body:**

```
{
  "company": "Google",
  "role": "Frontend Engineer",
  "status": "Interview",
  "location": "Bengaluru, India",
  "link": "https://careers.google.com/jobs",
  "interviewRounds": [
    { "name": "Technical Assessment", "status": "Done" },
    { "name": "System Design", "status": "Upcoming" }
  ]
}
```

```
]
}
```

- **Success Response (201 Created):** Returns the newly created database job object.

5. Dashboard Metrics & Front-end Integration

The dashboard homepage presents a unified overview of all resume assessment metrics, application status metrics, and goal progressions.

Endpoint: `GET /api/dashboard/overview`

Returns composite metrics parsed synchronously from historical databases:

- **Latest Resume Score & Delta:** Compares the score of the most recently uploaded resume against the previous document version, computing the rating difference (`delta`) to demonstrate progress tracking.
- **Application Success Funnel:** Computes the User's Interview Conversion Rate using the mathematical relation:

$$\text{Interview Conversion Rate (\%)} = (\text{Interviews} + \text{Offers}) / \text{Total Submitted Applications (Applied + Interview + Offer + Rejected)}$$

- **Weekly Target Counter:** Measures job application counts submitted over the past 7 days to calculate user progression progress towards their weekly goal.

Frontend State Synchronizer

The application's sidebar layout (`DashboardShell`) acts as a parent shell that handles application state.

- **Prefetching Engine:** Listens to mouse hover triggers (`onMouseEnter`) over navigation links (e.g. Dashboard, Resumes, Jobs) and pre-fetches resource datasets into the React Query cache before transitions happen. This creates a zero-delay transition user experience.
- **Mobile Optimizations:** Implements responsive gesture handlers (listening to touch gestures like `onTouchStart` , `onTouchMove` , and `onTouchEnd`) allowing users to close side drawer menus via left-swiping motions on handheld devices.

6. Installation & Configuration

To spin up the project environment locally or configure deployment builds on Vercel, the following steps must be completed.

Required Environment Keys (`.env`)

Variable Key	Description / Usage	Source
<code>NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY</code>	Clerk Identity SDK authorization key	Clerk Dashboard
<code>CLERK_SECRET_KEY</code>	Private signing key validating webhook requests	Clerk Dashboard
<code>DATABASE_URL</code>	Postgres Transaction connection pool URL	Supabase database setting
<code>DIRECT_URL</code>	Postgres Direct migration connection URL	Supabase database setting
<code>SUPABASE_URL</code>	Supabase project API endpoint domain	Supabase settings API page
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Bypasses RLS to read/write storage files	Supabase API settings (Secure!)
<code>GEMINI_API_KEY</code>	Accesses Google's generative models	Google AI Studio Developer Console
<code>GEMINI_MODEL</code>	Target Gemini model identifier	e.g., <code>gemini-1.5-flash</code>

Local Workspace Execution

1. Install packages:

```
npm install
```

2. Push Database Schema:

```
npx prisma db push
```

3. Launch Inngest development service to listen to local API endpoint:

```
npx inngest-cli@latest dev -u http://localhost:3000/api/inngest
```

4. Launch Next.js development server:

```
npm run dev
```

Prisma Post-Installation Webhook

The project is configured with a `postinstall` lifecycle script that runs `prisma generate` automatically on production deploys (e.g. Vercel builds), ensuring client type safety remains consistent without manual builds.